

Rodolfo Magana Lopez & Scott Iannaccone
Project 2
CPE 316, Professor Danowitz

Description:

For this project we made a waveform Generator using the STM and the DAC and the keypad. SPI communication was used to send a 12 bit digital value from the STM to the DAC which generated waveforms of sine, triangle, sawtooth, and square. TIM2 interrupts were used to control the DAC update timing instead of software delays, which helped us get frequencies of 100Hz to 500 Hz. This was used to generate the waves and the keypad allowed the user to choose the wave and the frequency they wanted, and change the Duty Cycle of the square wave.

Link to Video:

<https://www.youtube.com/watch?v=fZxD-8nv8kg>

Comments:

In the video The Oscilloscope Displays

- Voltage Max, Voltage Min
- Frequency, Duty Cycle

Our STM Code fits the requires of the following functions:

- Button 1 = 100Hz Freq
- Button 2 = 200Hz Freq
- Button 3 = 300Hz Freq
- Button 4 = 400Hz Freq
- Button 5 = 500Hz Freq
- Button 6 = SINE wave
- Button 7 = Triangle Wave
- Button 8 = Sawtooth Wave
- Button 9 = Square Wave
- Button * = Duty Cycle Decrease 10%
- Button 0 = 50% Square Wave Duty Cycle
- Button # = Duchy Cycle Increase 10%

Where the Duty Cycle only changes on Square Wave, and where all voltages have a $V_{pp} = 3V$.
With a DC Offset of 1.5V (Voltage Min will always be at 1.5V).

All of the following features can be seen tested in the Youtube video linked above.

Software Structure:

- The program re-uses a similar concept used in the KeyPad assignment that probes the Row pins to detect if the user pressed a button on the keypad with the column pins acting as inputs to the STM.
- The program uses SPI to communicate with the DAC. We reuse a volt_to_bits function from the DAC assignment to communicate with a DAC chip.
- The TIM2 timer is used. Provided below are the calculations for the ARR values for the corresponding Hz in frequency (100, 200, 300, 400, 500). It's a very 'back of napkin' calculation, but the values are shown to be very accurate on the Oscilloscope.
- For the Square Wave we have a built in square_wave function that uses some fancy math:
If we can represent Duty Cycle as 1, 2, 3, 4,
To correspond to 10%, 20%, 30%, 40%, ...
Then we can divide by 10 to get 0.1, 0.2, 0.3, ...

We can then do:

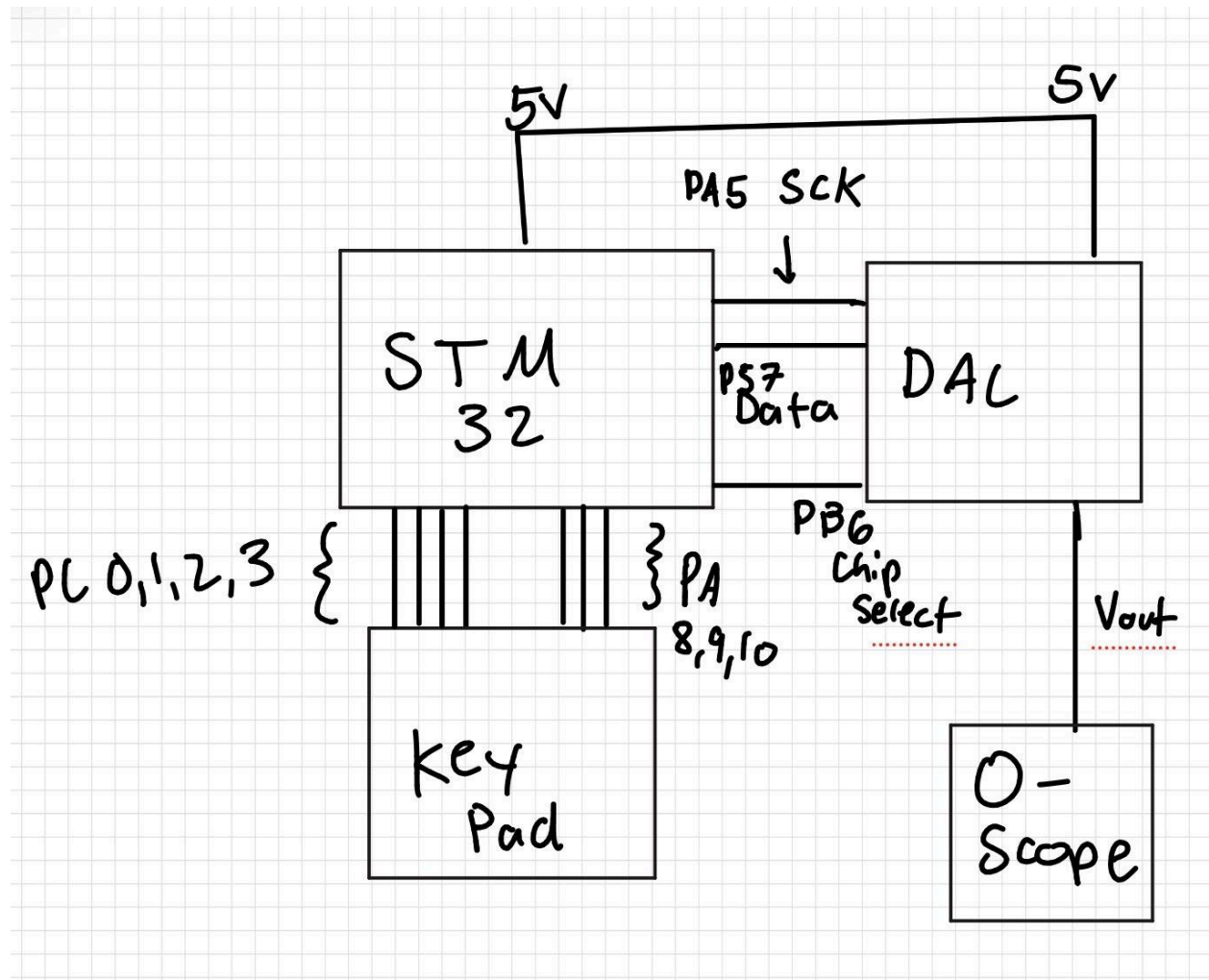
$$CCR1 = ARR - (ARR * (DutyCycle))$$

This will help us calculate the corresponding duty cycle square waves.

- SawTooth wave uses TIM2 in upcount mode and resets to zero on overflow. We use a value called 'sample freq' but this is really just a rolling CCR1. So we calculate CCR1 in Steps to get a nice wave, though if one were to zoom into the Oscilloscope they would be able to see the step transitions.
- Triangle waves Uses a similar concept above except the TIM2 is set to center mode meaning that the count starts to go back down after reaching ARR, back up when we reach zero, etc.
- Sine Wave uses a small but very effective premade table that was recommended in the Project Hints.
- 0 button only needs to change the ARR value to be set, set Duty Cycle to 5, and call the Square_wave function.

We ran into issues that we thought were debouncing, or logic priority, or maybe even race conditions, but it turns out it we plugged in wires to the wrong GPIO pins (it's always something like that).

Block Diagram:



SOURCE CODE

```
/* USER CODE END Header */
/* Includes -----*/
#include "main.h"
/* Private variables -----*/
#define DAC_CS_LOW()  GPIOB->BSRR = GPIO_BSRR_BR6
#define DAC_CS_HIGH() GPIOB->BSRR = GPIO_BSRR_BS6
/* KEEP SET A */
#define ROW1_PIN GPIO_PIN_0
#define ROW2_PIN GPIO_PIN_1
#define ROW3_PIN GPIO_PIN_2
#define ROW4_PIN GPIO_PIN_3
#define row_pins  (GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_3)
#define column_pins (GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_10)
#define WAVE_SINE 6
#define WAVE_TRIANGLE 7
#define WAVE_SAWTOOTH 8
```

```

#define WAVE_SQUARE      9
#define KEY_STAR  12
#define KEY_ZERO  0
#define KEY_HASH  15
#define SAMPLE_RATE 20000
#define FILTER_COUNT 3
UART_HandleTypeDef huart2;
/* Private function prototypes -----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USART2_UART_Init(void);
void square_wave(void);
void sawtooth_wave(void);
void triangle_wave(void);
void sine_wave(void);
void duty_inc(void);
void duty_dec(void);
void reset_tim2(void);
void DAC_write(uint16_t input);
uint16_t volt_to_bits(uint16_t voltage_mv);
void TIM2_IRQHandler(void);
void rows_off(void);
void drive_one_row(uint16_t row_pin);
void settle(void);
int columnCheck(void);
int keyPressed(int r, int c);
int read_keypad_raw(void);
int filter_keypad(void);
void handle_key(int key);
/* Global variables -----*/
volatile int ARR = 10000;
/* DUTY CYCLE: 10% = 1, 20% = 2, 30% = 3...., 90% = 9 */
volatile int duty_cycle = 5;
/* 6 = SINE, 7 = TRIANGLE, 8 = SAWTOOTH, 9 = SQUARE */
volatile unsigned char wave = WAVE_SQUARE;
/*
 * TIM2 runs at fixed 20 kHz sample rate.
 *
 * 100 Hz -> 200 samples/cycle
 * 200 Hz -> 100 samples/cycle
 * 300 Hz -> 67 samples/cycle, about 298.5 Hz
 * 400 Hz -> 50 samples/cycle
 * 500 Hz -> 40 samples/cycle
 */
volatile int samples_per_cycle = 200;
volatile int wave_index = 0;
/*
 * 32-point sine table.

```

```

* Output range: 1500 mV to 4500 mV.
*/
const uint16_t sine_table[32] =
{
    3000, 3293, 3574, 3833,
    4061, 4247, 4386, 4471,
    4500, 4471, 4386, 4247,
    4061, 3833, 3574, 3293,
    3000, 2707, 2426, 2167,
    1939, 1753, 1614, 1529,
    1500, 1529, 1614, 1753,
    1939, 2167, 2426, 2707
};

int main(void)
{
    int key;
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_USART2_UART_Init();
    /*
     * CONFIGURE KEYPAD
     * PC0-PC3 are open-drain row outputs.
     * PA8-PA10 are input columns with pull-ups.
     *
     * Rows idle HIGH/open.
     * Selected row is driven LOW.
     * Pressed key pulls a column LOW.
     */
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOCEN;
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
    /* PC0-PC3 output rows */
    GPIOC->MODER &= ~(GPIO_MODER_MODE0 |
                      GPIO_MODER_MODE1 |
                      GPIO_MODER_MODE2 |
                      GPIO_MODER_MODE3);
    GPIOC->MODER |= (GPIO_MODER_MODE0_0 |
                    GPIO_MODER_MODE1_0 |
                    GPIO_MODER_MODE2_0 |
                    GPIO_MODER_MODE3_0);

    /*
     * OPEN-DRAIN ROW OUTPUTS.
     * ODR = 1 means open / released.
     * ODR = 0 means actively pulling low.
     */
    GPIOC->OTYPER |= (GPIO_OTYPER_OT0 |
                     GPIO_OTYPER_OT1 |
                     GPIO_OTYPER_OT2 |

```

```

        GPIO_OTYPER_OT3);
GPIOC->PUPDR &= ~(GPIO_PUPDR_PUPD0 |
        GPIO_PUPDR_PUPD1 |
        GPIO_PUPDR_PUPD2 |
        GPIO_PUPDR_PUPD3);
GPIOC->OSPEEDR |= ((3 << GPIO_OSPEEDR_OSPEED0_Pos) |
        (3 << GPIO_OSPEEDR_OSPEED1_Pos) |
        (3 << GPIO_OSPEEDR_OSPEED2_Pos) |
        (3 << GPIO_OSPEEDR_OSPEED3_Pos));
rows_off();
/*
 * PA8-PA10 input columns with PULL-UPS.
 * No key pressed = column reads HIGH.
 * Pressed key on selected row = column reads LOW.
 */
GPIOA->MODER &= ~(GPIO_MODER_MODE8 |
        GPIO_MODER_MODE9 |
        GPIO_MODER_MODE10);
GPIOA->PUPDR &= ~(GPIO_PUPDR_PUPD8 |
        GPIO_PUPDR_PUPD9 |
        GPIO_PUPDR_PUPD10);
GPIOA->PUPDR |= (GPIO_PUPDR_PUPD8_0 |
        GPIO_PUPDR_PUPD9_0 |
        GPIO_PUPDR_PUPD10_0);

/* PC4 debug output */
GPIOC->PUPDR &= ~(GPIO_PUPDR_PUPD4);
GPIOC->MODER &= ~(GPIO_MODER_MODE4);
GPIOC->MODER |= GPIO_MODER_MODE4_0;
GPIOC->OTYPER &= ~(GPIO_OTYPER_OT4);
GPIOC->OSPEEDR |= (3 << GPIO_OSPEEDR_OSPEED4_Pos);
GPIOC->BRR = GPIO_PIN_4;
/* CONFIGURE DAC */
RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN | RCC_AHB2ENR_GPIOBEN;
RCC->APB2ENR |= RCC_APB2ENR_SPI1EN;
/* PA5 = SPI1 SCK, PA7 = SPI1 MOSI */
GPIOA->MODER &= ((3 << GPIO_MODER_MODE5_Pos) |
        (3 << GPIO_MODER_MODE7_Pos));
GPIOA->MODER |= ((2 << GPIO_MODER_MODE5_Pos) |
        (2 << GPIO_MODER_MODE7_Pos));
GPIOA->OSPEEDR |= ((3 << GPIO_OSPEEDR_OSPEED5_Pos) |
        (3 << GPIO_OSPEEDR_OSPEED7_Pos));
GPIOA->PUPDR &= ~(GPIO_PUPDR_PUPD5 | GPIO_PUPDR_PUPD7);
GPIOA->AFR[0] &= ((0xFU << GPIO_AFR_L_AFSEL5_Pos) |
        (0xFU << GPIO_AFR_L_AFSEL7_Pos));
GPIOA->AFR[0] |= ((5U << GPIO_AFR_L_AFSEL5_Pos) |
        (5U << GPIO_AFR_L_AFSEL7_Pos));

/* PB6 = DAC CS */
GPIOB->MODER &= ~(3 << GPIO_MODER_MODE6_Pos);

```

```

GPIOB->MODER |= (1 << GPIO_MODER_MODE6_Pos);
GPIOB->OTYPER &= ~(GPIO_OTYPER_OT6);
GPIOB->OSPEEDR |= (3 << GPIO_OSPEEDR_OSPEED6_Pos);
GPIOB->PUPDR &= ~(GPIO_PUPDR_PUPD6);
DAC_CS_HIGH();
/* Configure SPI1 */
SPI1->CR1 &= ~(SPI_CR1_SPE);
SPI1->CR1 = 0;
SPI1->CR2 = 0;
SPI1->CR1 |= SPI_CR1_MSTR |
             SPI_CR1_SSM |
             SPI_CR1_SSI |
             SPI_CR1_BR_1 |
             SPI_CR1_BR_0;
SPI1->CR2 |= (0xFU << SPI_CR2_DS_Pos);
SPI1->CR1 |= SPI_CR1_SPE;
DAC_write(volt_to_bits(1500));
/*
 * CONFIGURE TIM2
 *
 * System clock = 80 MHz.
 * PSC = 79 gives 1 MHz timer clock.
 * ARR = 49 gives 20 kHz update interrupt.
 *
 * TIM2 is the DAC sample timer.
 * CCR1 is not used.
 */
RCC->APB1ENR1 |= RCC_APB1ENR1_TIM2EN;
TIM2->CR1 = 0;
TIM2->PSC = 79;
TIM2->ARR = 49;
TIM2->CNT = 0;
TIM2->SR = 0;
TIM2->DIER = TIM_DIER_UIE;
NVIC->ISER[TIM2_IRQn / 32] |= (1 << (TIM2_IRQn % 32));
/* Power-up default: 100 Hz square wave, 50% duty */
ARR = 10000;
samples_per_cycle = 200;
duty_cycle = 5;
wave = WAVE_SQUARE;
wave_index = 0;
TIM2->CR1 |= TIM_CR1_CEN;
while (1)
{
    key = read_keypad_raw();
    if (key != -1)
    {
        GPIOC->BSRR = GPIO_PIN_4;
    }
}

```

```

        handle_key(key);
        HAL_Delay(150);
        GPIOC->BRR = GPIO_PIN_4;
        while (read_keypad_raw() != -1)
        {
            HAL_Delay(10);
        }
    }
    HAL_Delay(20);
}

/* ----- KEYPAD FUNCTIONS ----- */
/*
 * For open-drain outputs:
 * BSRR high = release row / high-Z
 */
void rows_off(void)
{
    GPIOC->BSRR = row_pins;
}

/*
 * For open-drain outputs:
 * First release all rows high.
 * Then pull selected row low.
 */
void drive_one_row(uint16_t row_pin)
{
    GPIOC->BSRR = row_pins;
    settle();
    GPIOC->BRR = row_pin;
}

void settle(void)
{
    volatile int i;
    for (i = 0; i < 5000; i++)
    {
    }
}

/*
 * Active-low column check.
 *
 * Columns have pull-ups.
 * No press = PA8, PA9, PA10 are high.
 * Pressed key = exactly one column goes low.
 */
int columnCheck(void)
{
    uint32_t low_cols;

```

```

low_cols = (~GPIOA->IDR) & column_pins;
if (low_cols == GPIO_PIN_8)
{
    return 1;
}
else if (low_cols == GPIO_PIN_9)
{
    return 2;
}
else if (low_cols == GPIO_PIN_10)
{
    return 3;
}
return -1;
}

int keyPressed(int r, int c)
{
    if (r == 1)
    {
        if (c == 1)
        {
            return 1;
        }
        else if (c == 2)
        {
            return 2;
        }
        else if (c == 3)
        {
            return 3;
        }
    }
    else if (r == 2)
    {
        if (c == 1)
        {
            return 4;
        }
        else if (c == 2)
        {
            return 5;
        }
        else if (c == 3)
        {
            return 6;
        }
    }
    else if (r == 3)

```

```

{
    if (c == 1)
    {
        return 7;
    }
    else if (c == 2)
    {
        return 8;
    }
    else if (c == 3)
    {
        return 9;
    }
}
else if (r == 4)
{
    if (c == 1)
    {
        return KEY_STAR;
    }
    else if (c == 2)
    {
        return KEY_ZERO;
    }
    else if (c == 3)
    {
        return KEY_HASH;
    }
}
return -1;
}
/*
 * Raw single-value keypad read.
 *
 * Return:
 * -1 = no valid key
 * 0 = key 0
 * 1-9 = keys 1-9
 * 12 = *
 * 15 = #
 *
 * Protection:
 * scans every row and only accepts exactly one detected key.
 */
int read_keypad_raw(void)
{
    int col;
    /*

```

```

    * Scan bottom-to-top.
    *
    * This gives priority to:
    * row 4: *, 0, #
    * row 3: 7, 8, 9
    *
    * That directly targets the problem where lower keypad rows
    * are getting ignored or shadowed by upper rows.
    */
    rows_off();
    drive_one_row(ROW4_PIN);
    settle();
    col = columnCheck();
    if (col != -1)
    {
        rows_off();
        return keyPressed(4, col);
    }
    rows_off();
    drive_one_row(ROW3_PIN);
    settle();
    col = columnCheck();
    if (col != -1)
    {
        rows_off();
        return keyPressed(3, col);
    }
    rows_off();
    drive_one_row(ROW2_PIN);
    settle();
    col = columnCheck();
    if (col != -1)
    {
        rows_off();
        return keyPressed(2, col);
    }
    rows_off();
    drive_one_row(ROW1_PIN);
    settle();
    col = columnCheck();
    if (col != -1)
    {
        rows_off();
        return keyPressed(1, col);
    }
    rows_off();
    return -1;
}

```

```
void handle_key(int key)
{
    if (key == 1)
    {
        ARR = 10000;
        samples_per_cycle = 200;
        wave_index = 0;
    }
    else if (key == 2)
    {
        ARR = 5000;
        samples_per_cycle = 100;
        wave_index = 0;
    }
    else if (key == 3)
    {
        ARR = 3333;
        samples_per_cycle = 67;
        wave_index = 0;
    }
    else if (key == 4)
    {
        ARR = 2500;
        samples_per_cycle = 50;
        wave_index = 0;
    }
    else if (key == 5)
    {
        ARR = 2000;
        samples_per_cycle = 40;
        wave_index = 0;
    }
    else if (key == 6)
    {
        sine_wave();
    }
    else if (key == 7)
    {
        triangle_wave();
    }
    else if (key == 8)
    {
        sawtooth_wave();
    }
    else if (key == 9)
    {
        square_wave();
    }
}
```

```

    else if (key == KEY_ZERO)
    {
        duty_cycle = 5;
        square_wave();
    }
    else if (key == KEY_STAR)
    {
        if (wave == WAVE_SQUARE)
        {
            duty_dec();
        }
    }
    else if (key == KEY_HASH)
    {
        if (wave == WAVE_SQUARE)
        {
            duty_inc();
        }
    }
}

/* ----- WAVEFORM FUNCTIONS ----- */
void square_wave(void)
{
    wave = WAVE_SQUARE;
    wave_index = 0;
}
void sine_wave(void)
{
    wave = WAVE_SINE;
    wave_index = 0;
}
void sawtooth_wave(void)
{
    wave = WAVE_SAWTOOTH;
    wave_index = 0;
}
void triangle_wave(void)
{
    wave = WAVE_TRIANGLE;
    wave_index = 0;
}
void duty_inc(void)
{
    if (duty_cycle < 9)
    {
        duty_cycle++;
    }
}
}

```

```

void duty_dec(void)
{
    if (duty_cycle > 1)
    {
        duty_cycle--;
    }
}

void reset_tim2(void)
{
    wave_index = 0;
}

/* ----- DAC FUNCTIONS ----- */
void DAC_write(uint16_t input)
{
    uint16_t cmd;
    cmd = input & 0xFFFF;
    cmd |= 0x3000;
    DAC_CS_LOW();
    while (!(SPI1->SR & SPI_SR_TXE))
    {
    }
    *(__IO uint16_t *)&SPI1->DR = cmd;
    while (SPI1->SR & SPI_SR_BSY)
    {
    }
    DAC_CS_HIGH();
}

uint16_t volt_to_bits(uint16_t voltage_mv)
{
    uint32_t value;
    if (voltage_mv > 4500)
    {
        voltage_mv = 4500;
    }
    else if (voltage_mv < 1500)
    {
        voltage_mv = 1500;
    }
    value = ((uint32_t)voltage_mv * 4095U) / 4500U;
    if (value > 4095U)
    {
        value = 4095U;
    }
    return (uint16_t)value;
}

/* ----- TIMER INTERRUPT HANDLER ----- */
void TIM2_IRQHandler(void)
{

```

```

uint16_t voltage_mv;
int half_cycle;
int threshold;
int sine_index;
if (TIM2->SR & TIM_SR_UIF)
{
    TIM2->SR &= ~(TIM_SR_UIF);
    if (samples_per_cycle < 2)
    {
        samples_per_cycle = 2;
    }
    if (wave_index >= samples_per_cycle)
    {
        wave_index = 0;
    }
    if (wave == WAVE_SQUARE)
    {
        threshold = (samples_per_cycle * duty_cycle) / 10;
        if (wave_index < threshold)
        {
            voltage_mv = 4500;
        }
        else
        {
            voltage_mv = 1500;
        }
    }
    else if (wave == WAVE_SAWTOOTH)
    {
        voltage_mv = 1500 + ((wave_index * 3000) / samples_per_cycle);
    }
    else if (wave == WAVE_TRIANGLE)
    {
        half_cycle = samples_per_cycle / 2;
        if (half_cycle < 1)
        {
            half_cycle = 1;
        }
        if (wave_index < half_cycle)
        {
            voltage_mv = 1500 + ((wave_index * 3000) / half_cycle);
        }
        else
        {
            voltage_mv = 4500 - (((wave_index - half_cycle) * 3000) / half_cycle);
        }
    }
    else if (wave == WAVE_SINE)

```

```

    {
        sine_index = (wave_index * 32) / samples_per_cycle;
        if (sine_index > 31)
        {
            sine_index = 31;
        }
        voltage_mv = sine_table[sine_index];
    }
    else
    {
        voltage_mv = 1500;
    }
    DAC_write(volt_to_bits(voltage_mv));
    wave_index++;
}
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
    if (HAL_PWREx_ControlVoltageScaling(PWR_REGULATOR_VOLTAGE_SCALE1) != HAL_OK)
    {
        Error_Handler();
    }
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSISState = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
    RCC_OscInitStruct.PLL.PLLM = 1;
    RCC_OscInitStruct.PLL.PLLN = 10;
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV7;
    RCC_OscInitStruct.PLL.PLLQ = RCC_PLLQ_DIV2;
    RCC_OscInitStruct.PLL.PLLR = RCC_PLLR_DIV2;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK |
                                   RCC_CLOCKTYPE_SYSCLK |
                                   RCC_CLOCKTYPE_PCLK1 |
                                   RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;

```

```

    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_4) != HAL_OK)
    {
        Error_Handler();
    }
}

/**
 * @brief USART2 Initialization Function
 * @param None
 * @retval None
 */
static void MX_USART2_UART_Init(void)
{
    huart2.Instance = USART2;
    huart2.Init.BaudRate = 115200;
    huart2.Init.WordLength = UART_WORDLENGTH_8B;
    huart2.Init.StopBits = UART_STOPBITS_1;
    huart2.Init.Parity = UART_PARITY_NONE;
    huart2.Init.Mode = UART_MODE_TX_RX;
    huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
    huart2.Init.OverSampling = UART_OVERSAMPLING_16;
    huart2.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE;
    huart2.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;
    if (HAL_UART_Init(&huart2) != HAL_OK)
    {
        Error_Handler();
    }
}

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    __HAL_RCC_GPIOC_CLK_ENABLE();
    __HAL_RCC_GPIOH_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();
    HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);
    GPIO_InitStruct.Pin = B1_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(B1_GPIO_Port, &GPIO_InitStruct);
    GPIO_InitStruct.Pin = LD2_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;

```

```
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(LD2_GPIO_Port, &GPIO_InitStruct);
}
/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    __disable_irq();
    while (1)
    {
    }
}
#ifdef USE_FULL_ASSERT
void assert_failed(uint8_t *file, uint32_t line)
{
}
#endif /* USE_FULL_ASSERT */
```